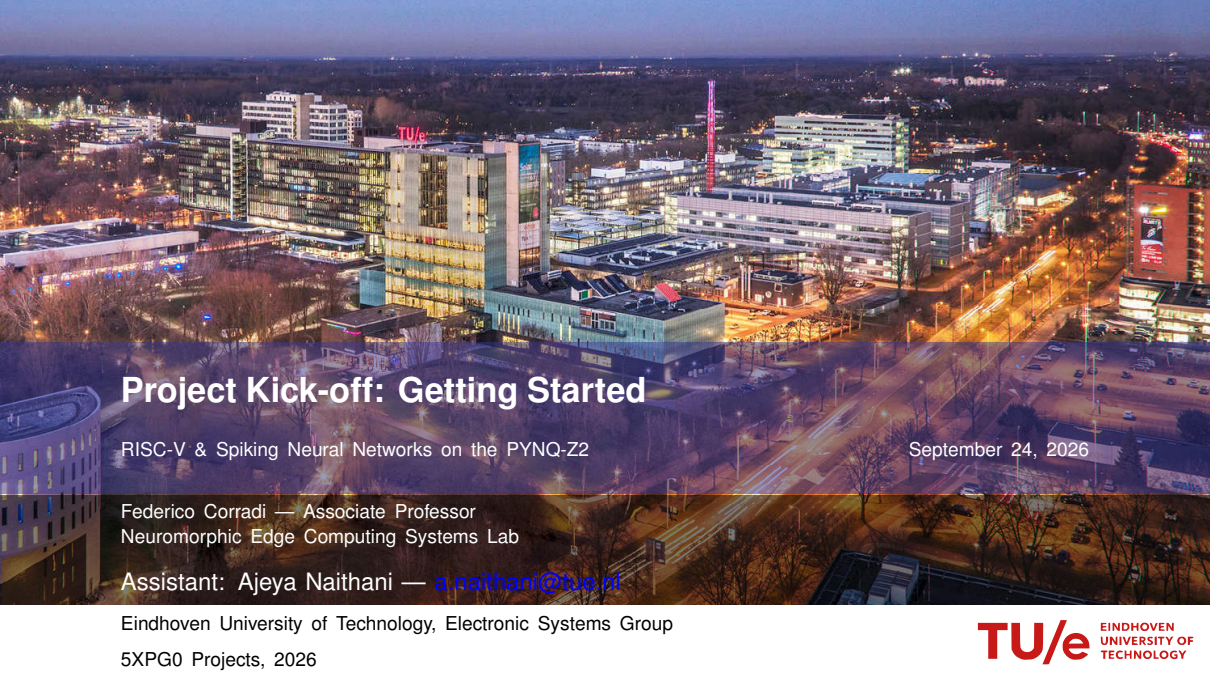




Project Kick-off: Getting Started

RISC-V & Spiking Neural Networks on the PYNQ-Z2

September 24, 2026

Federico Corradi — Associate Professor
Neuromorphic Edge Computing Systems Lab

Assistant: Ajeya Naithani — a.naithani@tue.nl

Eindhoven University of Technology, Electronic Systems Group

5XPG0 Projects, 2026

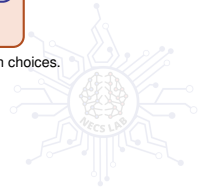
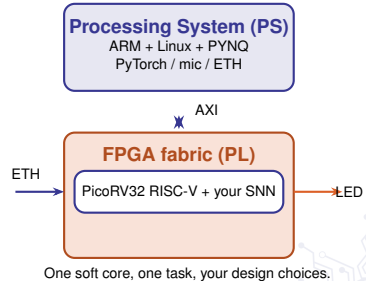
What you will build

Group 2 — single-keyword detection

- A **spiking neural network** for one keyword
- Deployed on the PYNQ-Z2 **RISC-V** core
- PC mic → spectrogram → board over ETH
- An **LED** lights for 1 s when the keyword is spoken

Group 4 — ANN vs SNN on MNIST

- One small **ANN**, one small **SNN**
- Both deployed on the **same RISC-V** core
- **Benchmark**: which one wins, and why?
- Extra: extend the ISA for SNNs



The two goals (one per group)

Group 2 — keyword spotter on a tiny core

Research goal: understand how **input encoding** and **network size** trade off against detection quality and latency, for a keyword detector running in **software** on a RISC-V core.

Demo goal: a microphone on the PC, a spectrogram sent over Ethernet, and an **LED that lights for 1 s** when the keyword is spoken.

Group 4 — ANN vs SNN, fairly

Research goal: under an **equal budget**, does a spiking network beat a classical ANN in **energy per inference** on a RISC-V core?

Demo goal: **both** classifiers run on the board, a **fair benchmark**, and a discussion. Extra: extend the ISA for the SNN.

The **research question** drives the work; the **demo** proves it. Both must be defended.

The building blocks — what the words mean

FPGA

A chip whose digital logic you **configure after manufacturing**. Here: a **Zynq-7020** = an ARM *processing system* (PS) plus **programmable logic** (PL). We load a design (a **bitstream**) onto the PL.

RISC-V

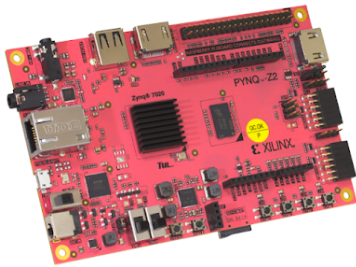
An **open, free instruction set**. We put the **PicoRV32** soft core (Verilog) on the FPGA and run C on it. Because it is open, you may **add instructions**.

SNN

A network that computes with **spikes over time** (leaky integrate-and-fire neurons) instead of real-valued activations. Event-driven, potentially very low power. An **ANN** is the classical, non-spiking network.

In this project the network runs in **software** on the RISC-V core; moving it into the FPGA fabric is the advanced step.

Your platform & the one prerequisite



PYNQ-Z2 (Zynq-7020, 512 MB DDR3)



You need one PYNQ-Z2, prepared once

- Stock PYNQ 3.1 SD image, Ethernet cable
- SSH key auth for user `xilinx`: `ssh xilinx@<board-ip>` must work with no password
- Passwordless `sudo` on the board — stock credentials are `xilinx / xilinx`
- PC and board on the same network; board can reach the internet

We give you a ready-to-run flow

- A prebuilt bitstream + the full Vivado build flow
- The RISC-V firmware and host tools, with comments
- A `snnTorch` training example (one epoch, deliberately small)

What is in the 5XPG0 repo

Code & tools

- `pynqz2_riscv_flow/` — RISC-V + Poisson + one LIF neuron
 - `install.sh`, `run_demo.sh`, `prebuilt.bit`
 - `rtl/`, `firmware/`, `vivado/`, `host/`
- `snn_keyword/` — Group 2 training skeleton
- `ann_vs_snn_mnist/` — Group 4 training skeleton
- These slides (PDF) + a README in every folder

Every folder explains itself

- `README.md` in `pynqz2_riscv_flow/` — the flow
- `README.md` in each group folder — the task + plan

Start in three commands

- `./install.sh <board-ip>`
- `./run_demo.sh <board-ip>`
- `make bitstream` (optional, needs Vivado 2024.1)

The skeleton is not the solution

It runs, but it is small, under-trained and full of design choices we left open for you.

Using AI — read this carefully

Allowed

- You may use AI tools for **everything**: code, reading, debugging, writing, slides.
- Coding and group work together is **absolutely allowed**.

Your responsibility

- You are responsible for the results — not the AI.
- You must understand, verify and defend every choice.
- A logical gap or a claim you cannot explain costs you.

Use AI as a fast assistant, not as the author of your science.

Method: design first, execute second

Spend your time here

- Formulate a sharp **research question**.
- State a **hypothesis** before you implement.
- Design the experiment so it can **disprove** you.
- Only then implement and execute.

Many open design choices

- input encoding
- network topology (under HW constraints)
- training method
- demo setup (network + LED)
- ... and how you benchmark

Weekly rhythm

Monday 08:30–09:00 and Thursday 13:30–14:00
(both groups, every week — come prepared)

33%

Research-question
formulation

33%

Scientific execution
(no logical gaps)

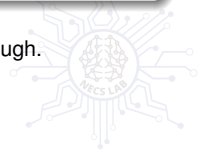
33%

Engineering solutions
(results)

Deliverables

Report · Code · Presentation

All three categories weigh the same — a beautiful demo with no question is not enough.



New to FPGA, RISC-V or SNNs? Start here.

FPGA / PYNQ

- PYNQ docs: <https://pynq.readthedocs.io>
- Nandland intro: <https://www.nandland.com>
- Read `rtl/poisson.v` — small and commented

RISC-V

- <https://riscv.org> — the open ISA
- PicoRV32 core: <https://github.com/cliffordwolf/picorv32>
- The firmware is plain C in `firmware/main.c`

Spiking neural networks

- snnTorch: <https://snntorch.readthedocs.io>
- Neuronal Dynamics (free book): <https://neurondynamics.epfl.ch>
- Training code in the two group folders

You are **not** expected to know these yet — learn as you go and ask questions in the Monday / Thursday meetings.

Where to look for help

Start here

- [These slides](#) (the kick-off PDF) — how to get started
- `code/pynqz2_riscv_flow/README.md` — the hardware flow
- The two group READMEs — training + plan
- Every source file is commented

Background reading

- F. Corradi et al., *Gyro: a digital SNN architecture...*, 2021
- A. Sankaran et al., *An event-driven recurrent SNN architecture...*, ICONS 2022
- H. Irmak et al., *A dynamic reconfigurable architecture...*, JLPEA 2021

Federico Corradi f.corradi@tue.nl · Ajeya Naithani a.naithani@tue.nl

Talk to us on Monday or Thursday.